

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

#1. Titanic survival prediction

```
df=pd.read_csv('/content/tested.csv')
df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3
...	...	...	...	...	...	...	...	...	
413	1305	0	3	Spector, Mr. Woolf	male	NaN	0	0	A.
414	1306	1	1	Oliva y Ocana, Dona. Fermina	female	39.0	0	0	PO
415	1307	0	3	Saether, Mr. Simon Sivertsen	male	38.5	0	0	SOTC 3
416	1308	0	3	Ware, Mr. Frederick	male	NaN	0	0	
417	1309	0	3	Peter, Master. Michael J	male	NaN	1	1	
418 rows × 12 columns									

```
# a. Data cleaning
```

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Cabin'].fillna(df['Cabin'].mode()[0], inplace=True)
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  418 non-null    int64
1   Survived     418 non-null    int64
2   Pclass       418 non-null    int64
3   Name         418 non-null    object
4   Sex          418 non-null    object
5   Age          418 non-null    float64
6   SibSp        418 non-null    int64
7   Parch        418 non-null    int64
8   Ticket       418 non-null    object
9   Fare         417 non-null    float64
10  Cabin        418 non-null    object
11  Embarked     418 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 39.3+ KB
/tmp/ipykernel_1191/542644663.py:1: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using

    df['Age'].fillna(df['Age'].mean(), inplace=True)
/tmp/ipykernel_1191/542644663.py:2: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using

    df['Cabin'].fillna(df['Cabin'].mode()[0], inplace=True)

```

b. Exploratory Data Analysis

```

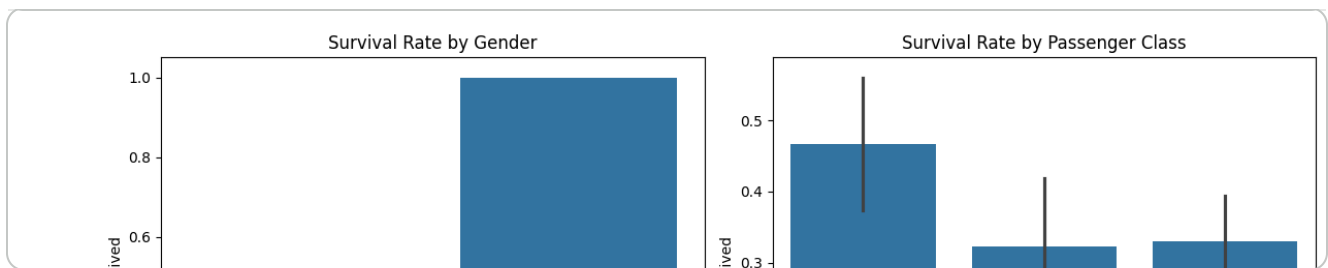
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
sns.barplot(x='Sex', y='Survived', data=df)
plt.title('Survival Rate by Gender')

plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
sns.barplot(x='Pclass', y='Survived', data=df)
plt.title('Survival Rate by Passenger Class')

plt.tight_layout()
plt.show()

```

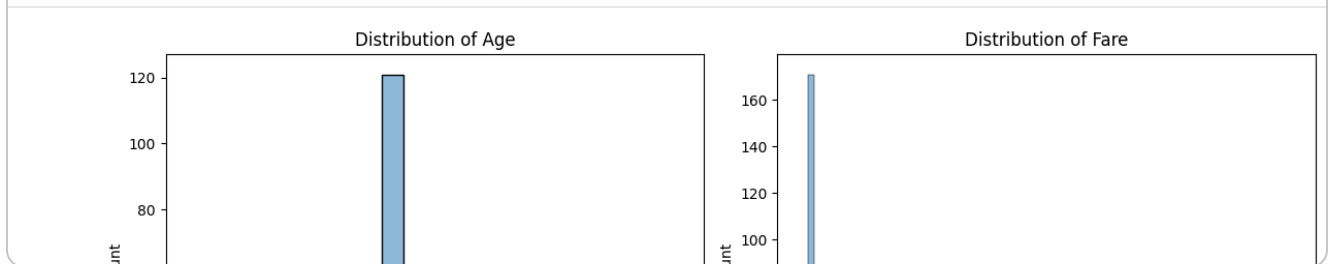


```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
sns.histplot(df['Age'], kde=True)
plt.title('Distribution of Age')
plt.xlabel('Age')
plt.ylabel('Count')

plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
sns.histplot(df['Fare'], kde=True)
plt.title('Distribution of Fare')
plt.xlabel('Fare')
plt.ylabel('Count')

plt.tight_layout()
plt.show()
```



#c. Feature Engineering

```
df['FamilySize'] = df['SibSp'] + df['Parch']
display(df.head())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticke
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	33091
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	36327
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	24027
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	31515
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	310129

```
df['AgeGroup'] = pd.cut(df['Age'], bins=[0, 12, 18, 60, 100], labels=['Child',
display(df.head())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	33091
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	36327
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	24027
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	31515
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	310129

#d. Model building

```

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Assuming X_train, X_test, y_train, y_test are already defined in the environment

# 1. Logistic Regression
print("\n--- Logistic Regression ---")
log_reg_model = LogisticRegression(random_state=42, solver='liblinear')
log_reg_model.fit(X_train, y_train)
y_pred_lr = log_reg_model.predict(X_test)

accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

print(f"Accuracy: {accuracy_lr:.4f}")
print(f"Precision: {precision_lr:.4f}")
print(f"Recall: {recall_lr:.4f}")
print(f"F1-score: {f1_lr:.4f}")

```

```

# 2. Decision Tree Classifier
print("\n--- Decision Tree Classifier ---")
dec_tree_model = DecisionTreeClassifier(random_state=42)
dec_tree_model.fit(X_train, y_train)
y_pred_dt = dec_tree_model.predict(X_test)

accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)

print(f"Accuracy: {accuracy_dt:.4f}")
print(f"Precision: {precision_dt:.4f}")
print(f"Recall: {recall_dt:.4f}")
print(f"F1-score: {f1_dt:.4f}")

# 3. Random Forest Classifier
print("\n--- Random Forest Classifier ---")
rand_forest_model = RandomForestClassifier(random_state=42)
rand_forest_model.fit(X_train, y_train)
y_pred_rf = rand_forest_model.predict(X_test)

accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

print(f"Accuracy: {accuracy_rf:.4f}")
print(f"Precision: {precision_rf:.4f}")
print(f"Recall: {recall_rf:.4f}")
print(f"F1-score: {f1_rf:.4f}")

```

```

--- Logistic Regression ---
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

--- Decision Tree Classifier ---
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

--- Random Forest Classifier ---
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

```

#e. Evaluation

```

from sklearn.model_selection import StratifiedKFold, cross_val_score

# Assuming X and y are already defined in the environment

```

```

# Setup Stratified K-Fold for robust evaluation
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("\n--- Cross-Validation Results ---")

# 1. Logistic Regression with Cross-Validation
print("\nLogistic Regression (Cross-Validation):")
log_reg_cv_model = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)

scoring = ['accuracy', 'precision', 'recall', 'f1']

for metric in scoring:
    cv_scores = cross_val_score(log_reg_cv_model, X, y, cv=kfold, scoring=metric)
    print(f" {metric.capitalize()}: Mean = {cv_scores.mean():.4f}, Std = {cv_scores.std():.4f}")

# 2. Decision Tree Classifier with Cross-Validation
print("\nDecision Tree Classifier (Cross-Validation):")
dec_tree_cv_model = DecisionTreeClassifier(random_state=42)

for metric in scoring:
    cv_scores = cross_val_score(dec_tree_cv_model, X, y, cv=kfold, scoring=metric)
    print(f" {metric.capitalize()}: Mean = {cv_scores.mean():.4f}, Std = {cv_scores.std():.4f}")

# 3. Random Forest Classifier with Cross-Validation
print("\nRandom Forest Classifier (Cross-Validation):")
rand_forest_cv_model = RandomForestClassifier(random_state=42)

for metric in scoring:
    cv_scores = cross_val_score(rand_forest_cv_model, X, y, cv=kfold, scoring=metric)
    print(f" {metric.capitalize()}: Mean = {cv_scores.mean():.4f}, Std = {cv_scores.std():.4f}")

```

--- Cross-Validation Results ---

Logistic Regression (Cross-Validation):
 Accuracy: Mean = 1.0000, Std = 0.0000
 Precision: Mean = 1.0000, Std = 0.0000
 Recall: Mean = 1.0000, Std = 0.0000
 F1: Mean = 1.0000, Std = 0.0000

Decision Tree Classifier (Cross-Validation):
 Accuracy: Mean = 1.0000, Std = 0.0000
 Precision: Mean = 1.0000, Std = 0.0000
 Recall: Mean = 1.0000, Std = 0.0000
 F1: Mean = 1.0000, Std = 0.0000

Random Forest Classifier (Cross-Validation):
 Accuracy: Mean = 1.0000, Std = 0.0000
 Precision: Mean = 1.0000, Std = 0.0000
 Recall: Mean = 1.0000, Std = 0.0000
 F1: Mean = 1.0000, Std = 0.0000

#2. Iris Flower Classification

#a. Data load


```
df=pd.read_csv('/content/iris_data.csv')
df
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

Data Exploration

```
print("Missing values in each column:")
df.isnull().sum()
```

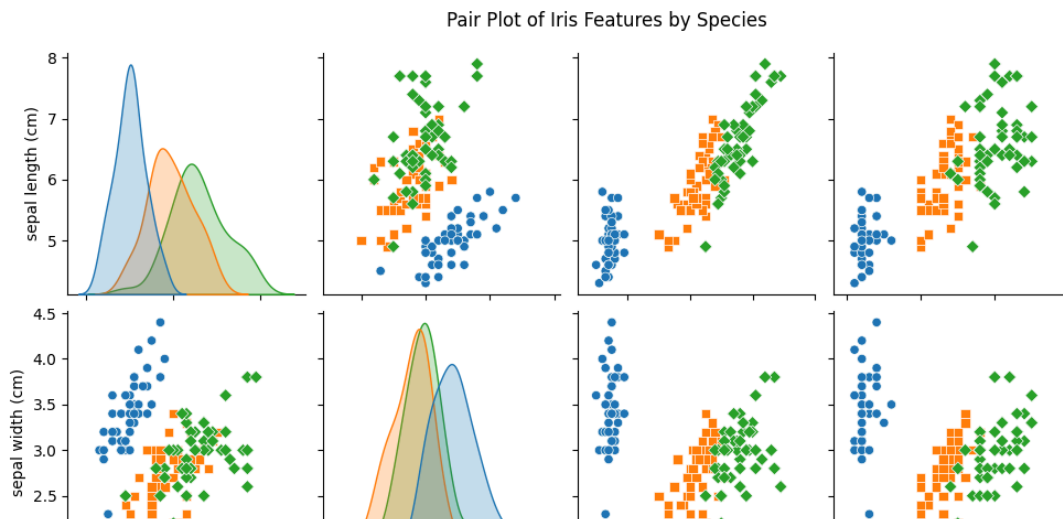
Missing values in each column:

	0
sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
Species	0

dtype: int64

#b. Visualization

```
sns.pairplot(df, hue='Species', markers=["o", "s", "D"])
plt.suptitle('Pair Plot of Iris Features by Species', y=1.02)
plt.show()
```



#c. Model building

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Separate features (X) and target (y) for the Iris dataset
X_iris = df.drop('Species', axis=1)
y_iris = df['Species']

# Encode target variable if it's not numeric (e.g., 'setosa', 'versicolor', 'virginica')
# This is handled automatically by many sklearn classifiers, but explicitly encoding
# For simplicity, let's assume it will be handled by the models or ensure it's
# If y_iris is strings, LabelEncoder can be used:
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_iris_encoded = le.fit_transform(y_iris)

# Split data into training and testing sets
X_train_iris, X_test_iris, y_train_iris, y_test_iris = train_test_split(X_iris, y_iris_encoded,
                                                                           test_size=0.3,
                                                                           random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled_iris = scaler.fit_transform(X_train_iris)
X_test_scaled_iris = scaler.transform(X_test_iris)

print("\n--- Iris Classification Model Building ---")

# 1. Logistic Regression
print("\nLogistic Regression:")
log_reg_iris_model = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)
log_reg_iris_model.fit(X_train_scaled_iris, y_train_iris)
y_pred_lr_iris = log_reg_iris_model.predict(X_test_scaled_iris)
```

```

accuracy_lr_iris = accuracy_score(y_test_iris, y_pred_lr_iris)
print(f"Accuracy: {accuracy_lr_iris:.4f}")
print("Classification Report:\n", classification_report(y_test_iris, y_pred_lr_iris))

# 2. Support Vector Machine (SVM)
print("\nSupport Vector Machine (SVM):")
svm_iris_model = SVC(random_state=42)
svm_iris_model.fit(X_train_scaled_iris, y_train_iris)
y_pred_svm_iris = svm_iris_model.predict(X_test_scaled_iris)
accuracy_svm_iris = accuracy_score(y_test_iris, y_pred_svm_iris)
print(f"Accuracy: {accuracy_svm_iris:.4f}")
print("Classification Report:\n", classification_report(y_test_iris, y_pred_svm_iris))

# 3. K-Nearest Neighbors (KNN)
print("\nK-Nearest Neighbors (KNN):")
knn_iris_model = KNeighborsClassifier(n_neighbors=5) # n_neighbors is a common
knn_iris_model.fit(X_train_scaled_iris, y_train_iris)
y_pred_knn_iris = knn_iris_model.predict(X_test_scaled_iris)
accuracy_knn_iris = accuracy_score(y_test_iris, y_pred_knn_iris)
print(f"Accuracy: {accuracy_knn_iris:.4f}")
print("Classification Report:\n", classification_report(y_test_iris, y_pred_knn_iris))

```

--- Iris Classification Model Building ---

Logistic Regression:

Accuracy: 0.8000

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.75	0.60	0.67	15
virginica	0.67	0.80	0.73	15
accuracy			0.80	45
macro avg	0.81	0.80	0.80	45
weighted avg	0.81	0.80	0.80	45

Support Vector Machine (SVM):

Accuracy: 0.9333

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45

K-Nearest Neighbors (KNN):

Accuracy: 0.9111

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

setosa	1.00	1.00	1.00	15
versicolor	0.79	1.00	0.88	15
virginica	1.00	0.73	0.85	15
accuracy			0.91	45
macro avg	0.93	0.91	0.91	45
weighted avg	0.93	0.91	0.91	45

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py
warnings.warn(
```

#d. Evaluation

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Assuming y_test_iris, y_pred_lr_iris, y_pred_svm_iris, y_pred_knn_iris, and ...

print("\n--- Confusion Matrices for Iris Classification ---")

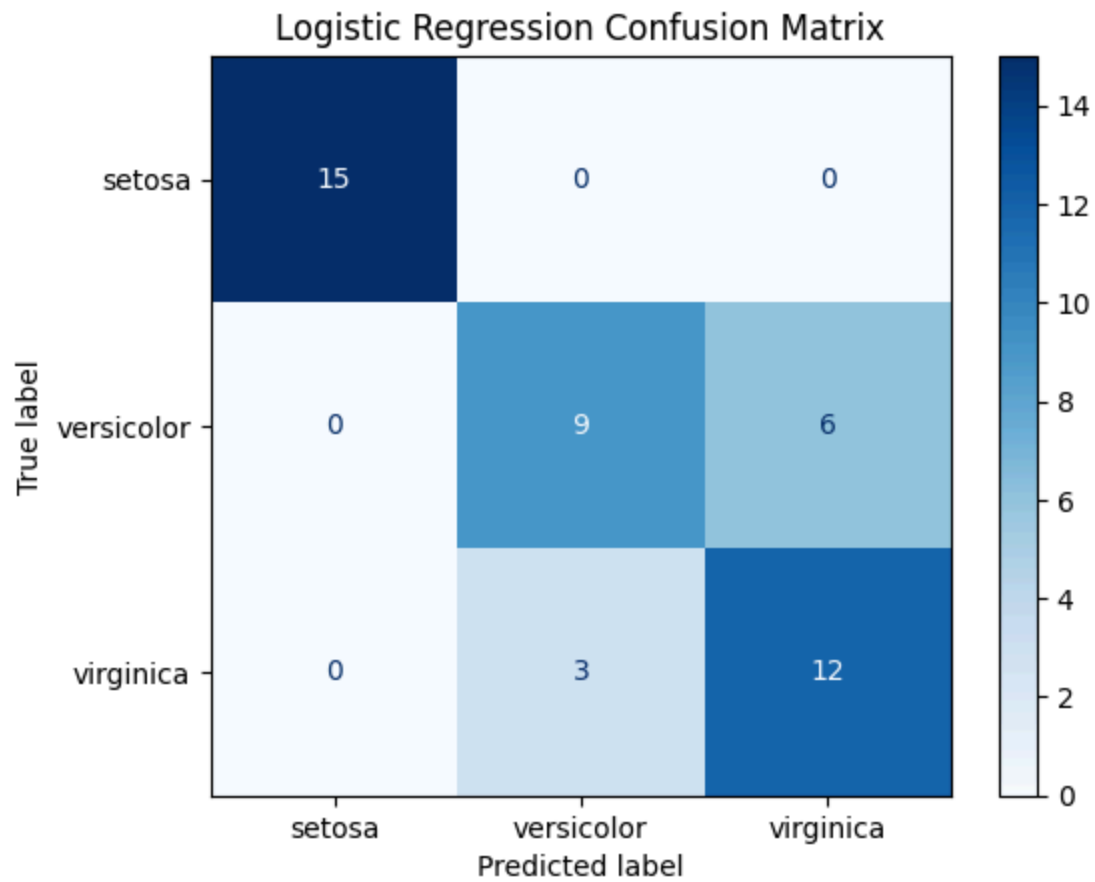
# Confusion Matrix for Logistic Regression
print("\nLogistic Regression Confusion Matrix:")
cm_lr = confusion_matrix(y_test_iris, y_pred_lr_iris)
disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=le.classes_)
disp_lr.plot(cmap=plt.cm.Blues)
plt.title('Logistic Regression Confusion Matrix')
plt.show()

# Confusion Matrix for Support Vector Machine (SVM)
print("\nSupport Vector Machine (SVM) Confusion Matrix:")
cm_svm = confusion_matrix(y_test_iris, y_pred_svm_iris)
disp_svm = ConfusionMatrixDisplay(confusion_matrix=cm_svm, display_labels=le.classes_)
disp_svm.plot(cmap=plt.cm.Blues)
plt.title('SVM Confusion Matrix')
plt.show()

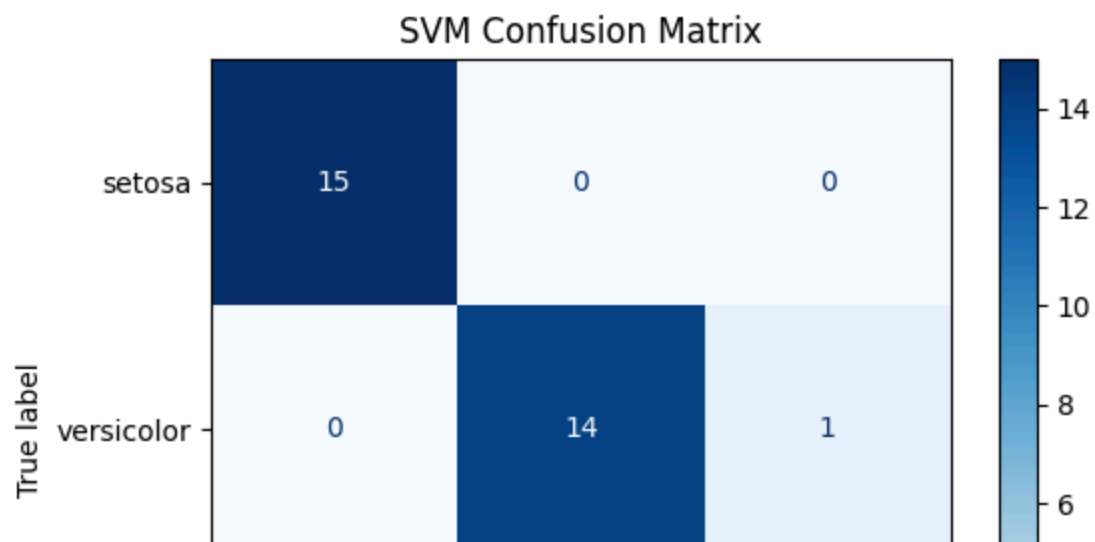
# Confusion Matrix for K-Nearest Neighbors (KNN)
print("\nK-Nearest Neighbors (KNN) Confusion Matrix:")
cm_knn = confusion_matrix(y_test_iris, y_pred_knn_iris)
disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn, display_labels=le.classes_)
disp_knn.plot(cmap=plt.cm.Blues)
plt.title('KNN Confusion Matrix')
plt.show()
```

--- Confusion Matrices for Iris Classification ---

Logistic Regression Confusion Matrix:



Support Vector Machine (SVM) Confusion Matrix:



#3. House price prediction

load dataset

```
df=pd.read_csv('/content/data.csv')
df
```

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
0	2014-05-02 00:00:00	3.130000e+05	3.0	1.50	1340	7912	
1	2014-05-02 00:00:00	2.384000e+06	5.0	2.50	3650	9050	
2	2014-05-02 00:00:00	3.420000e+05	3.0	2.00	1930	11947	
3	2014-05-02 00:00:00	4.200000e+05	3.0	2.25	2000	8030	
4	2014-05-02 00:00:00	5.500000e+05	4.0	2.50	1940	10500	
...	...	...	...	...	...	...	
4595	2014-07-09 00:00:00	3.081667e+05	3.0	1.75	1510	6360	
4596	2014-07-09 00:00:00	5.343333e+05	3.0	2.50	1460	7573	
4597	2014-07-09 00:00:00	4.169042e+05	3.0	2.50	3010	7014	
4598	2014-07-10 00:00:00	2.034000e+05	4.0	2.00	2090	6630	
4599	2014-07-10 00:00:00	2.206000e+05	3.0	2.50	1490	8102	

4600 rows × 8 columns

```
#a. Data cleaning
```

```
print("Missing values before cleaning:")
df.info()
```

```

Missing values before cleaning:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                   4600 non-null   object
1   price                  4600 non-null   float64
2   bedrooms               4600 non-null   float64
3   bathrooms              4600 non-null   float64
4   sqft_living            4600 non-null   int64
5   sqft_lot               4600 non-null   int64
6   floors                 4600 non-null   float64
7   waterfront             4600 non-null   int64
8   view                   4600 non-null   int64
9   condition              4600 non-null   int64
10  sqft_above             4600 non-null   int64
11  sqft_basement          4600 non-null   int64
12  yr_built                4600 non-null   int64
13  yr_renovated           4600 non-null   int64
14  street                 4600 non-null   object
15  city                   4600 non-null   object
16  statezip               4600 non-null   object
17  country                4600 non-null   object
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB

```

```

# Fill missing numerical values with the mean
for col in ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors', 'sqft_basement']:
    if df[col].isnull().any():
        df[col].fillna(df[col].mean(), inplace=True)

# Fill missing categorical values with the mode
for col in ['waterfront', 'view', 'condition', 'street', 'city', 'statezip', 'country']:
    if df[col].isnull().any():
        df[col].fillna(df[col].mode()[0], inplace=True)

print("\nMissing values after cleaning:")
df.info()

```

```

Missing values after cleaning:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                   4600 non-null   object
1   price                  4600 non-null   float64
2   bedrooms               4600 non-null   float64
3   bathrooms              4600 non-null   float64
4   sqft_living            4600 non-null   int64
5   sqft_lot               4600 non-null   int64
6   floors                 4600 non-null   float64
7   waterfront             4600 non-null   int64

```

```

8   view          4600 non-null   int64
9   condition     4600 non-null   int64
10  sqft_above    4600 non-null   int64
11  sqft_basement 4600 non-null   int64
12  yr_built      4600 non-null   int64
13  yr_renovated  4600 non-null   int64
14  street        4600 non-null   object
15  city          4600 non-null   object
16  statezip      4600 non-null   object
17  country       4600 non-null   object
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB

```

```

print("Descriptive Statistics:")
print(df.describe())

```

Descriptive Statistics:

	price	bedrooms	bathrooms	sqft_living	sqft_lot
count	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e+03
mean	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e+04
std	5.638347e+05	0.908848	0.783781	963.206916	3.588444e+04
min	0.000000e+00	0.000000	0.000000	370.000000	6.380000e+02
25%	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e+03
50%	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e+03
75%	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e+04
max	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e+06

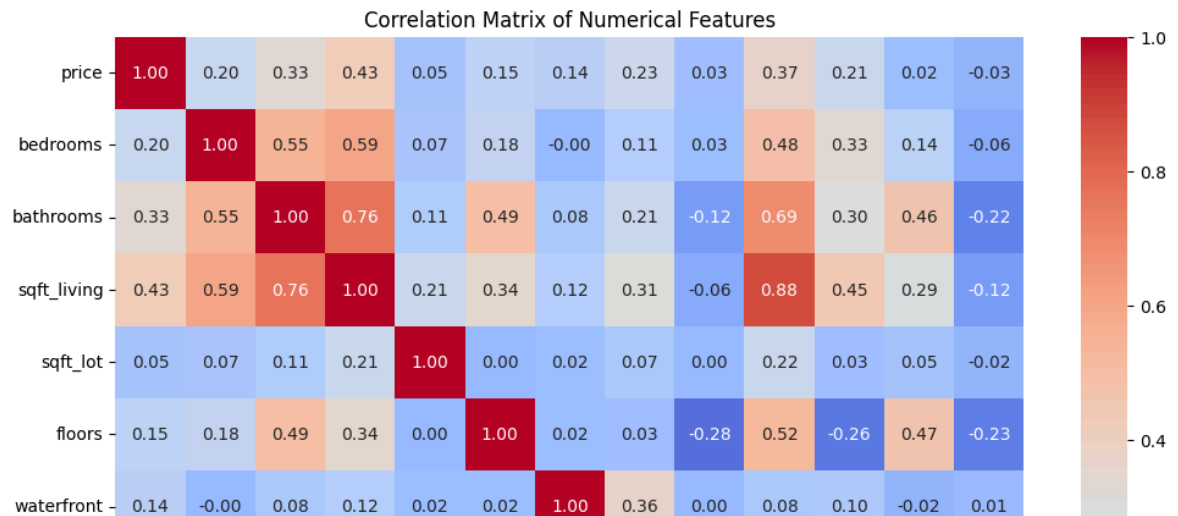
	floors	waterfront	view	condition	sqft_above
count	4600.000000	4600.000000	4600.000000	4600.000000	4600.000000
mean	1.512065	0.007174	0.240652	3.451739	1827.265435
std	0.538288	0.084404	0.778405	0.677230	862.168977
min	1.000000	0.000000	0.000000	1.000000	370.000000
25%	1.000000	0.000000	0.000000	3.000000	1190.000000
50%	1.500000	0.000000	0.000000	3.000000	1590.000000
75%	2.000000	0.000000	0.000000	4.000000	2300.000000
max	3.500000	1.000000	4.000000	5.000000	9410.000000

	sqft_basement	yr_built	yr_renovated
count	4600.000000	4600.000000	4600.000000
mean	312.081522	1970.786304	808.608261
std	464.137228	29.731848	979.414536
min	0.000000	1900.000000	0.000000
25%	0.000000	1951.000000	0.000000
50%	0.000000	1976.000000	0.000000
75%	610.000000	1997.000000	1999.000000
max	4820.000000	2014.000000	2014.000000

```

plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features')
plt.show()

```

```
# Feature Engineering
if 'date' in df.columns:
    df['date'] = pd.to_datetime(df['date'])
    df['year'] = df['date'].apply(lambda x: x.year)
    df['month'] = df['date'].apply(lambda x: x.month)
    df.drop('date', axis=1, inplace=True)

# One-hot encode categorical features ('street', 'city', 'statezip', 'country')
categorical_cols = ['street', 'city', 'statezip', 'country']
cols_to_encode = [col for col in categorical_cols if col in df.columns]

if cols_to_encode:
    df = pd.get_dummies(df, columns=cols_to_encode, drop_first=True)

print("DataFrame after Feature Engineering:")
df.head()
```

DataFrame after Feature Engineering:

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront
0	313000.0	3.0	1.50	1340	7912	1.5	0
1	2384000.0	5.0	2.50	3650	9050	2.0	0
2	342000.0	3.0	2.00	1930	11947	1.0	0
3	420000.0	3.0	2.25	2000	8030	1.0	0
4	550000.0	4.0	2.50	1940	10500	1.0	0

5 rows x 4658 columns

#b. Exploratory Data Analysis

```
plt.figure(figsize=(15, 10))
```

```
plt.subplot(2, 3, 1)
sns.scatterplot(x='sqft_living', y='price', data=df)
plt.title('House Price vs. Living Area')

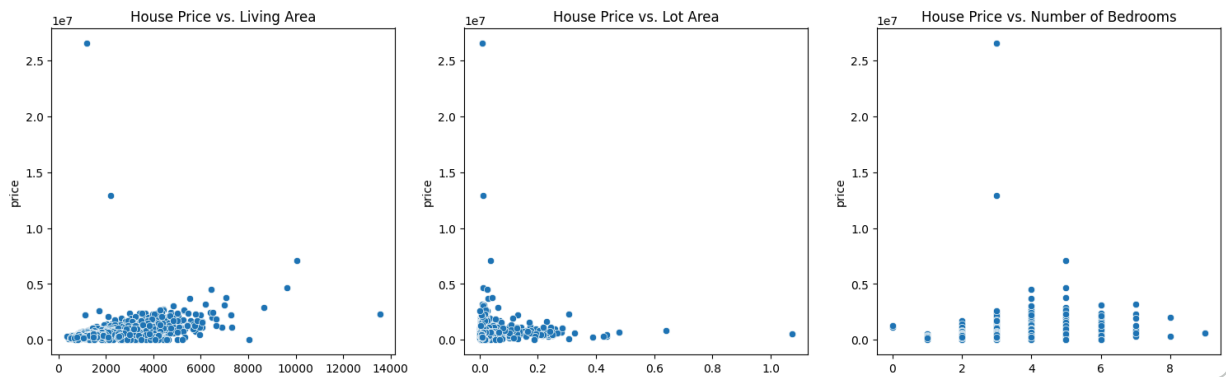
plt.subplot(2, 3, 2)
sns.scatterplot(x='sqft_lot', y='price', data=df)
plt.title('House Price vs. Lot Area')

plt.subplot(2, 3, 3)
sns.scatterplot(x='bedrooms', y='price', data=df)
plt.title('House Price vs. Number of Bedrooms')

plt.subplot(2, 3, 4)
sns.scatterplot(x='bathrooms', y='price', data=df)
plt.title('House Price vs. Number of Bathrooms')

plt.subplot(2, 3, 5)
sns.scatterplot(x='floors', y='price', data=df)
plt.title('House Price vs. Number of Floors')

plt.tight_layout()
plt.show()
```



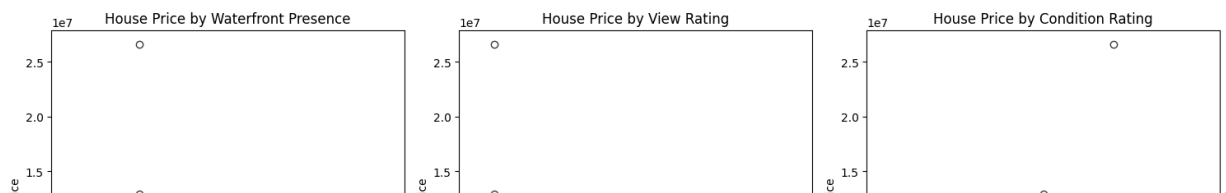
```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(x='waterfront', y='price', data=df)
plt.title('House Price by Waterfront Presence')

plt.subplot(1, 3, 2)
sns.boxplot(x='view', y='price', data=df)
plt.title('House Price by View Rating')

plt.subplot(1, 3, 3)
sns.boxplot(x='condition', y='price', data=df)
plt.title('House Price by Condition Rating')

plt.tight_layout()
plt.show()
```



#c. Feature Engineering

```
df['total_sqft'] = df['sqft_living'] + df['sqft_lot']
df['house_age'] = df['year'] - df['yr_built']
df['years_since_renovation'] = df['year'] - df['yr_renovated']

# For properties that were never renovated (yr_renovated is 0), set years_since_renovation to house_age
df.loc[df['yr_renovated'] == 0, 'years_since_renovation'] = df['house_age']

display(df[['total_sqft', 'house_age', 'years_since_renovation']].head())
```

	total_sqft	house_age	years_since_renovation
0	7919.201171	59	9
1	9058.202756	93	93
2	11954.565793	48	48
3	8037.601402	51	51
4	10507.570959	38	22

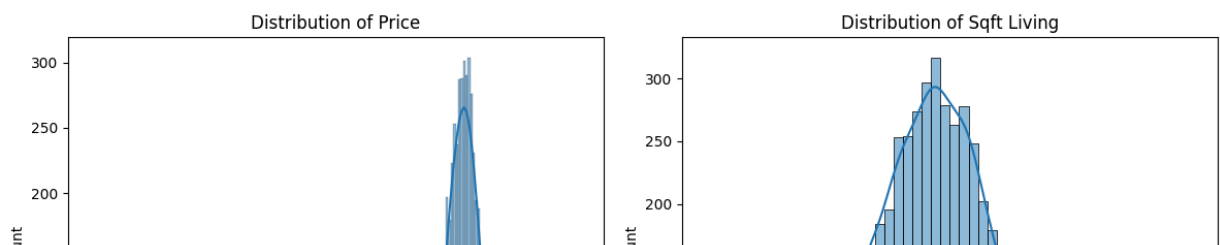
Now, let's check the distribution of `price` and `sqft_living` to see if they are skewed and might benefit from a log transformation.

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(df['price'], kde=True)
plt.title('Distribution of Price')

plt.subplot(1, 2, 2)
sns.histplot(df['sqft_living'], kde=True)
plt.title('Distribution of Sqft Living')

plt.tight_layout()
plt.show()
```



Both 'price' and 'sqft_living' appear to be right-skewed. Let's apply a log transformation to these features to make their distributions more normal, which can improve model performance.

```

df['price'] = np.log1p(df['price'])
df['sqft_living'] = np.log1p(df['sqft_living'])

print("Distributions after log transformation:")

plt.figure(figsize=(12, 5))

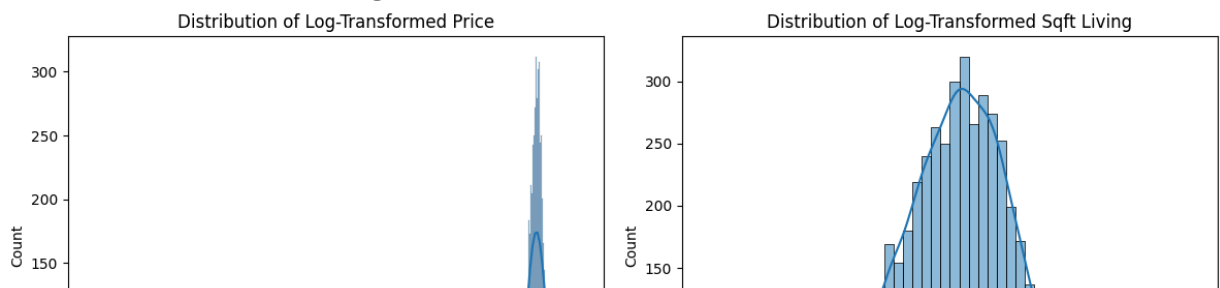
plt.subplot(1, 2, 1)
sns.histplot(df['price'], kde=True)
plt.title('Distribution of Log-Transformed Price')

plt.subplot(1, 2, 2)
sns.histplot(df['sqft_living'], kde=True)
plt.title('Distribution of Log-Transformed Sqft Living')

plt.tight_layout()
plt.show()

```

Distributions after log transformation:



✓ d. Model Building - Data Preparation

```

# Drop non-numeric and irrelevant columns for model training
# 'street', 'city', 'statezip', 'country' were one-hot encoded previously. 'date'
# 'yr_built' and 'yr_renovated' were used to create 'house_age' and 'years_since'
# 'year' and 'month' were also created. Let's keep them.

# Identify columns to drop
columns_to_drop = ['yr_built', 'yr_renovated'] # These were used to create age

# Ensure 'date' is dropped if it still exists and was not already converted to
# In the previous `feature engineering` step, `date` column was already dropped
# We also already one-hot encoded 'street', 'city', 'statezip', 'country', so

# Let's verify the columns in df after all previous operations
# The current df has a lot of one-hot encoded columns, and `yr_built` and `yr_
# The `date` column was already dropped.

X = df.drop(['price'] + columns_to_drop, axis=1, errors='ignore')
y = df['price']

# Check for any remaining non-numeric columns that might have been missed or c
non_numeric_cols = X.select_dtypes(include=['object', 'bool']).columns
if len(non_numeric_cols) > 0:
    print(f"Warning: Dropping non-numeric columns: {list(non_numeric_cols)}")
    X = X.drop(columns=non_numeric_cols, axis=1)

from sklearn.model_selection import train_test_split

```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

X_train.head()
```

```
Warning: Dropping non-numeric columns: ['street_10 W Etruria St', 'street_11 W Etruria St']
Shape of X_train: (3680, 15)
Shape of X_test: (920, 15)
Shape of y_train: (3680,)
Shape of y_test: (920,)
```

	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view
1898	4.0	2.50	2.189076	45514	2.0	0	0
1370	4.0	3.00	2.221565	29043	2.0	0	0
3038	4.0	2.50	2.190680	11120	2.0	0	0
2361	4.0	3.75	2.230205	10800	2.0	0	0
156	3.0	2.00	2.151925	7000	2.0	0	0

✓ e. Model Training and Evaluation for House Price Prediction

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

print("\n--- House Price Prediction Model Training & Evaluation ---")

# List of models to train
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(random_state=42),
    'Lasso Regression': Lasso(random_state=42),
    'Random Forest Regressor': RandomForestRegressor(random_state=42)
}

# Train and evaluate each model
results = {}

for name, model in models.items():
    print(f"\nTraining {name}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    # Evaluate the model
    mse = mean_squared_error(y_test, y_pred)
```

```

rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

results[name] = {'MSE': mse, 'RMSE': rmse, 'R2': r2}

print(f" Mean Squared Error (MSE): {mse:.4f}")
print(f" Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f" R-squared (R2): {r2:.4f}")

# Display a summary of results
print("\n--- Model Performance Summary ---")
for name, metrics in results.items():
    print(f"{name}:")
    for metric_name, value in metrics.items():
        print(f" {metric_name}: {value:.4f}")

```

--- House Price Prediction Model Training & Evaluation ---

Training Linear Regression...

```

Mean Squared Error (MSE): 0.0692
Root Mean Squared Error (RMSE): 0.2630
R-squared (R2): 0.0003

```

Training Ridge Regression...

```

Mean Squared Error (MSE): 0.0691
Root Mean Squared Error (RMSE): 0.2630
R-squared (R2): 0.0005

```

Training Lasso Regression...

```

Mean Squared Error (MSE): 0.0693
Root Mean Squared Error (RMSE): 0.2632
R-squared (R2): -0.0016

```

Training Random Forest Regressor...

```

Mean Squared Error (MSE): 0.0856
Root Mean Squared Error (RMSE): 0.2925
R-squared (R2): -0.2369

```

--- Model Performance Summary ---

Linear Regression:

```

MSE: 0.0692
RMSE: 0.2630
R2: 0.0003

```

Ridge Regression:

```

MSE: 0.0691
RMSE: 0.2630
R2: 0.0005

```

Lasso Regression:

```

MSE: 0.0693
RMSE: 0.2632
R2: -0.0016

```

Random Forest Regressor:

```

MSE: 0.0856
RMSE: 0.2925
R2: -0.2369

```

#4. Customer Segmentation with Clustering

```
# load dataset
```

```
df=pd.read_csv('/content/Mall_Customers.csv')  
df
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

200 rows × 5 columns

#a. Data preprocessing

```
from sklearn.preprocessing import StandardScaler  
  
# Select the features for clustering: Annual Income and Spending Score  
X_customer = df[['Annual Income (k$)', 'Spending Score (1-100)']]  
  
# Scale the features  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_customer)  
  
# Create a DataFrame from the scaled features for easier inspection (optional)  
X_scaled_df = pd.DataFrame(X_scaled, columns=X_customer.columns)  
display(X_scaled_df.head())
```

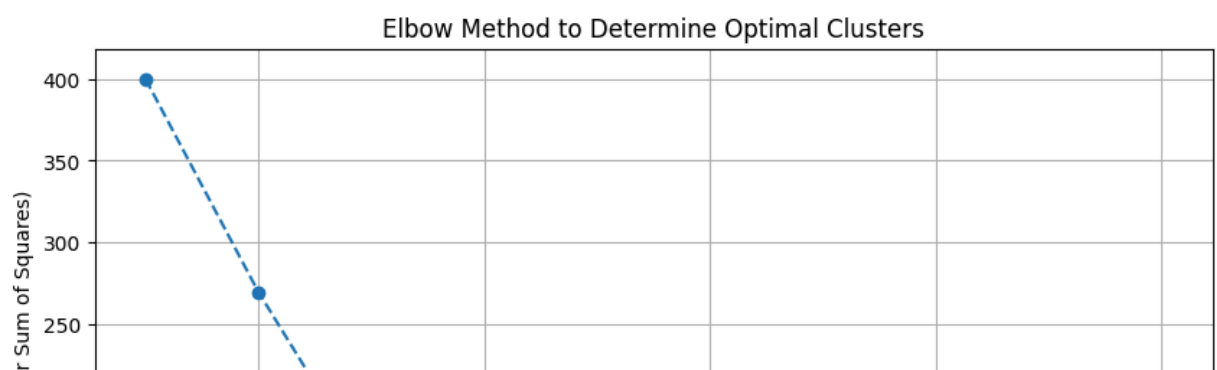
	Annual Income (k\$)	Spending Score (1-100)
0	-1.738999	-0.434801
1	-1.738999	1.195704
2	-1.700830	-1.715913
3	-1.700830	1.040418
4	-1.662660	-0.395980

#b. Clustering

```
from sklearn.cluster import KMeans

wcss = []
# Try different numbers of clusters from 1 to 10
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the Elbow Method graph
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method to Determine Optimal Clusters')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.grid(True)
plt.show()
```



```
n_clusters_optimal = 5 # Based on visual inspection of the Elbow Method plot
kmeans_optimal = KMeans(n_clusters=n_clusters_optimal, init='k-means++', random_state=42)
y_kmeans = kmeans_optimal.fit_predict(X_scaled)

# Add the cluster labels to the original (unscaled) DataFrame for analysis
df['Cluster'] = y_kmeans

# Visualize the clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', hue='Cluster')
```

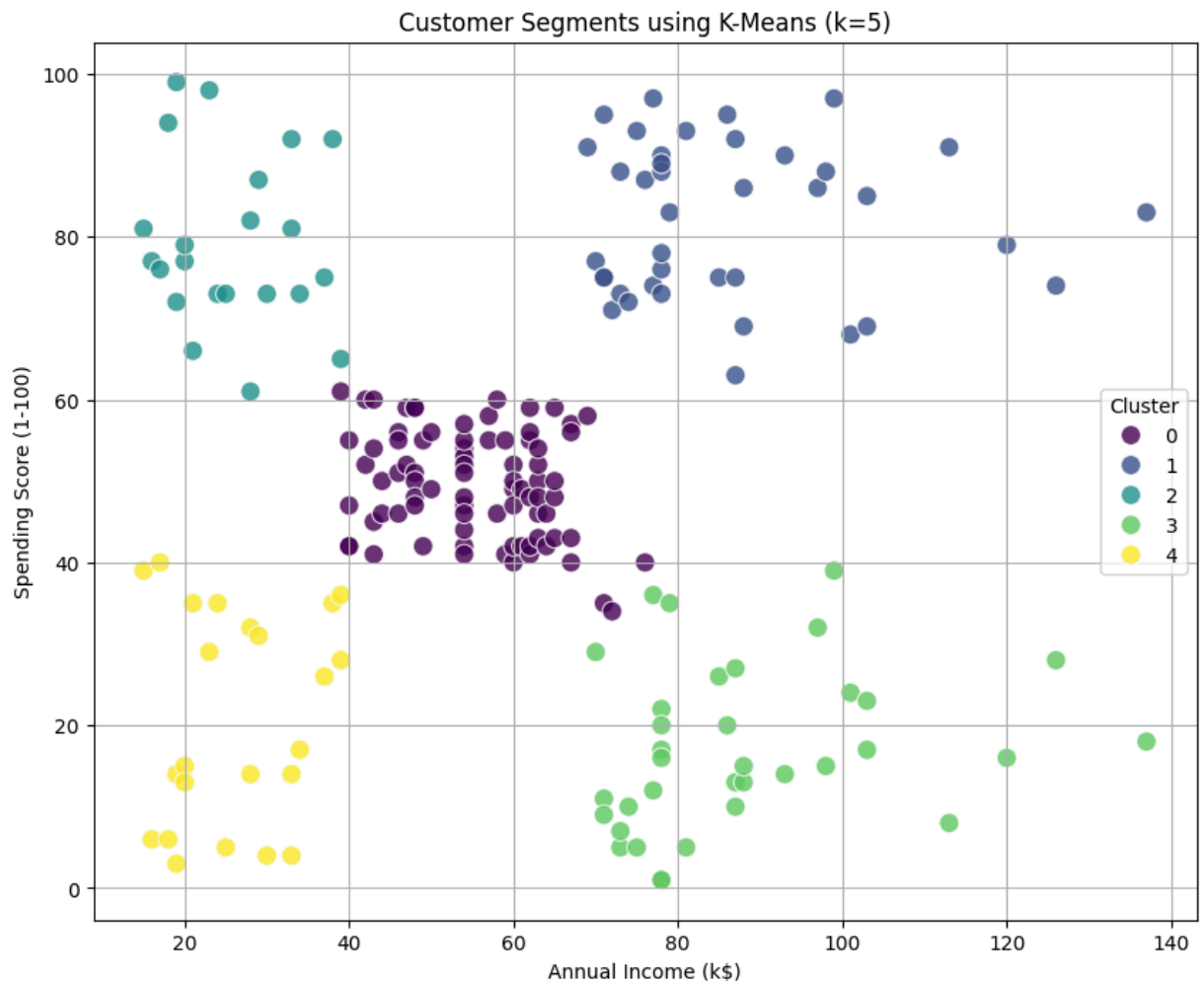


```

        palette='viridis', s=100, alpha=0.8)
plt.title(f'Customer Segments using K-Means (k={n_clusters_optimal})')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.grid(True)
plt.legend(title='Cluster')
plt.show()

# Display the characteristics of each cluster
print("\nCluster Characteristics (Mean Values):")
display(df.groupby('Cluster')[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].mean())

```



Cluster Characteristics (Mean Values):

#c. Visualization

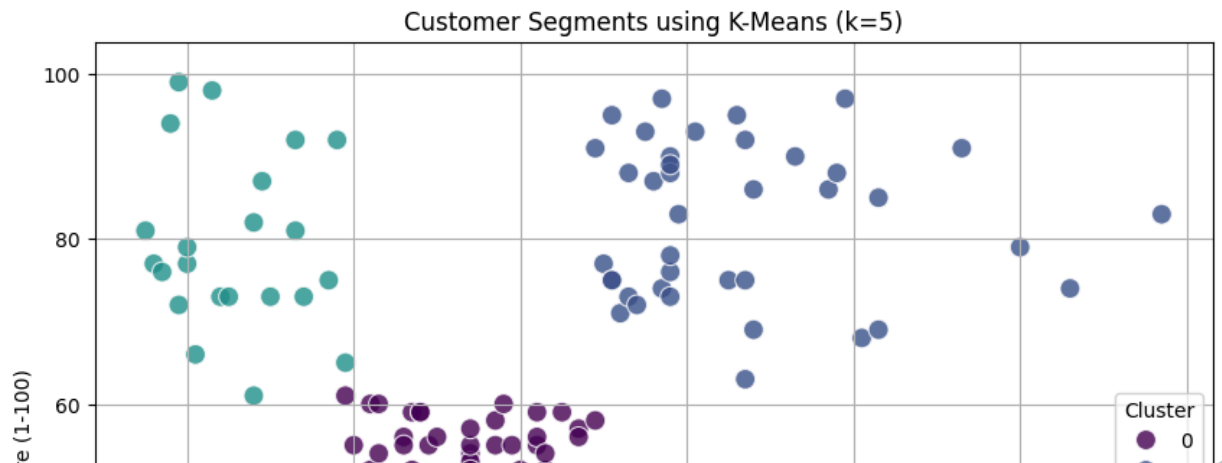
```

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 8))
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', hue='Cluster',
                palette='viridis', s=100, alpha=0.8)
plt.title('Customer Segments using K-Means (k=5)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')

```

```
plt.grid(True)
plt.legend(title='Cluster')
plt.show()
```



#d. Interpretation:

✓ Interpreting the Customer Segments

Based on the K-Means clustering with 5 segments, we can characterize each cluster by examining the mean values of 'Age', 'Annual Income (k\$)', and 'Spending Score (1-100)':

- **Cluster 0 (Moderate/Average Shoppers):**

- **Average Age:** ~42.7 years
- **Average Annual Income:** ~\$55.3k
- **Average Spending Score:** ~49.5
- This segment represents a typical customer base with moderate income and spending habits. They are middle-aged and form a significant portion of the mall's visitors.

- **Cluster 1 (High Income, High Spending - 'Target Customers'):**

- **Average Age:** ~32.7 years
- **Average Annual Income:** ~\$86.5k
- **Average Spending Score:** ~82.1
- This is a highly desirable segment, consisting of relatively younger adults with high incomes and a strong propensity to spend. They are likely trend-conscious and seek premium products or experiences.

- **Cluster 2 (Low Income, High Spending - 'Budget-Conscious Spenders'):**

- **Average Age:** ~25.3 years
- **Average Annual Income:** ~\$25.7k

- **Average Spending Score:** ~79.4
- This segment includes the youngest customers, who, despite having lower incomes, exhibit high spending scores. They might be students or young professionals who prioritize certain products or experiences, possibly being more impulsive or seeking value for money that aligns with their interests.
- **Cluster 3 (High Income, Low Spending - 'Frugal High-Earners'):**
 - **Average Age:** ~41.1 years
 - **Average Annual Income:** ~\$88.2k
 - **Average Spending Score:** ~17.1
 - These are middle-aged, high-income individuals who are very conservative in their spending at the mall. They might visit for specific needs but are not driven by impulse or luxury purchases within the mall setting.
- **Cluster 4 (Low Income, Low Spending - 'Thrifty Seniors'):**
 - **Average Age:** ~45.2 years
 - **Average Annual Income:** ~\$26.3k
 - **Average Spending Score:** ~20.9
 - This segment comprises older customers with both low income and low spending. They are likely budget-conscious, practical shoppers who only purchase necessities or highly discounted items.

```
#5. Movie Recommendation System
```

```
#a. Data load
```

```
df=pd.read_csv('/content/ratings.csv')  
df
```

	user_id	movie_id	rating	timestamp
0	33	34	5	2023-01-01
1	24	18	4	2023-01-02
2	75	13	4	2023-01-03
3	72	41	5	2023-01-04
4	36	23	3	2023-01-05
...	...	...	...	...
1650	77	42	5	2028-06-15
1651	31	26	2	2028-06-16
1652	68	10	5	2028-06-18
1653	98	45	3	2028-06-19
1654	19	11	3	2028-06-21

1655 rows × 4 columns

```
# preprocessing
```

✓ Create User-Item Matrix

```
user_movie_matrix = df.pivot_table(index='user_id', columns='movie_id', values='rating')
display(user_movie_matrix.head())
```

movie_id	1	2	3	4	5	6	7	8	9	10	...	41	42
user_id													
1	NaN	NaN	5.0	4.0	NaN	NaN	NaN	4.0	5.0	NaN	...	NaN	NaN
2	4.0	4.0	2.0	4.0	NaN	NaN	NaN	3.0	2.0	NaN	...	5.0	NaN
3	NaN	NaN	NaN	3.0	4.0	NaN	3.0	NaN	5.0	NaN	...	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.0	NaN
5	4.0	NaN	NaN	NaN	NaN	NaN	4.0	NaN	NaN	5.0	...	4.0	NaN

5 rows × 50 columns

```
#b. Collaborative Filtering
```

✓ Item-Based Collaborative Filtering

For item-based collaborative filtering, we need to calculate the similarity between movies. We'll use cosine similarity for this, treating unrated movies as having a rating of 0 during similarity calculation. This is a common approach when dealing with sparse rating matrices.

```
from sklearn.metrics.pairwise import cosine_similarity

# Fill NaN values with 0 for similarity calculation
# This assumes unrated movies don't contribute to similarity, or that a 0 rating
# For prediction, we will only consider actual ratings.
movie_similarity = cosine_similarity(user_movie_matrix.fillna(0).T)

movie_similarity_df = pd.DataFrame(movie_similarity, index=user_movie_matrix.columns, columns=user_movie_matrix.columns)
display(movie_similarity_df.head())
```

movie_id	1	2	3	4	5	6	7	8
movie_id								
1	1.000000	0.386867	0.272282	0.363192	0.171486	0.271436	0.493866	0.271436
2	0.386867	1.000000	0.402263	0.443364	0.313019	0.309444	0.178688	0.309444
3	0.272282	0.402263	1.000000	0.383628	0.281309	0.222125	0.266368	0.402263
4	0.363192	0.443364	0.383628	1.000000	0.414823	0.264750	0.329721	0.363192
5	0.171486	0.313019	0.281309	0.414823	1.000000	0.247945	0.221730	0.247945

5 rows × 50 columns

✓ Movie Recommendation Function

Now, let's create a function that takes a `user_id` as input and recommends movies based on the item-item similarity. The recommendation logic will:

1. Get the movies rated by the user.
2. For each rated movie, find its similar movies.
3. Calculate a weighted score for each potential recommendation based on the user's rating for the similar movie and the similarity between the two movies.
4. Filter out movies already rated by the user.
5. Return the top N recommendations.

```

def recommend_movies(user_id, num_recommendations=5):
    # Get the user's ratings from the user-movie matrix
    user_ratings = user_movie_matrix.loc[user_id].dropna()

    if user_ratings.empty:
        return f"User {user_id} has not rated any movies yet."

    # Initialize a dictionary to store recommendation scores
    recommendation_scores = {}

    for movie_id, rating in user_ratings.items():
        # Get similar movies for the current rated movie
        # Exclude the movie itself from similarity calculation (similarity to self is 1)
        similar_movies = movie_similarity_df[movie_id].drop(movie_id)

        for similar_movie_id, similarity_score in similar_movies.items():
            # Only consider movies not yet rated by the user
            if similar_movie_id not in user_ratings.index:
                # Weighted score: user's rating * similarity score
                # This weights recommendations higher if the user liked a similar movie
                if similar_movie_id not in recommendation_scores:
                    recommendation_scores[similar_movie_id] = 0
                recommendation_scores[similar_movie_id] += rating * similarity_score

    # Sort recommendations by score in descending order
    sorted_recommendations = sorted(recommendation_scores.items(), key=lambda x: x[1], reverse=True)

    # Get top N movie IDs
    top_movie_ids = [movie_id for movie_id, score in sorted_recommendations[:num_recommendations]]

    return top_movie_ids

# Example usage:
user_to_recommend = 1
recommendations = recommend_movies(user_to_recommend)
print(f"Top movie recommendations for User {user_to_recommend}: {recommendations}")

user_to_recommend = 50
recommendations = recommend_movies(user_to_recommend)
print(f"Top movie recommendations for User {user_to_recommend}: {recommendations}")

```

```

Top movie recommendations for User 1: [2, 36, 47, 39, 17]
Top movie recommendations for User 50: [4, 11, 49, 45, 32]

```

#6. Sentiment Analysis of Tweets

#a. Load dataset(AI)

```

df=pd.read_csv('/content/tweets_data_unified-3.csv')
df

```

	tweet_id	user_id	username	user_location	user_followers	user_
0	61	1092	CloudGeek	Sydney, Australia	300604	
1	22	1064	TechVisionary	Berlin, Germany	121075	
2	40	1047	CloudGeek	Austin, TX	380102	
3	75	1054	StartupFounder	London, UK	94279	
4	10	1075	CyberSecure	Amsterdam, Netherlands	132974	
...	...	...	...	...	...	
70	3	1015	RoboticsFan	London, UK	84996	
71	49	1073	DataScientist	Stockholm, Sweden	321284	
72	12	1100	DigitalTransformer	Sydney, Australia	466972	
73	21	1002	AIResearcher	Toronto, Canada	309087	
74	31	1091	DataScientist	Mumbai, India	365971	

75 rows × 28 columns

#b. Data Cleaning

```

import re

def clean_tweet_text(text):
    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    # Remove mentions
    text = re.sub(r'@\w+', '', text)
    # Remove hashtags
    text = re.sub(r'#\w+', '', text)
    # Remove special characters and numbers, keep alphanumeric and spaces
    text = re.sub(r'^a-zA-Z\s]', '', text)
    # Convert to lowercase
    text = text.lower()
    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df['cleaned_text'] = df['text'].apply(clean_tweet_text)
print("Original vs Cleaned Text Samples:")
print(df[['text', 'cleaned_text']].head())

```

Original vs Cleaned Text Samples:

	text	cleaned_text
0	AI algorithms perpetuating systemic racism in ...	ai algorithms perpetuating systemic racism in ...
1	AI generated art is stealing from human artist...	ai generated art is stealing from human artist...
2	MIT researchers develop new AI safety framewor...	mit researchers develop new ai safety framewor...
3	AI-powered recommendation systems are actually...	aipowered recommendation systems are actually ...
4	Job interviews evaluated by AI? Sounds like di...	job interviews evaluated by ai sounds like dis...

#c. Text Preprocessing

```

import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Ensure NLTK's data path includes the default download location for Colab
nltk.data.path.append('/root/nltk_data')

# Download necessary NLTK data (if not already downloaded)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt_tab') # Explicitly download to prevent LookupError

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    # Tokenize the text
    tokens = nltk.word_tokenize(text)

```



```

# Remove stop words and lemmatize
processed_tokens = [lemmatizer.lemmatize(word) for word in tokens if word != '']
# Join tokens back into a string
return ' '.join(processed_tokens)

df['processed_text'] = df['cleaned_text'].apply(preprocess_text)

print("Cleaned vs Processed Text Samples:")
print(df[['cleaned_text', 'processed_text']].head())

```

Cleaned vs Processed Text Samples:

```

              cleaned_text \
0  ai algorithms perpetuating systemic racism in ...
1  ai generated art is stealing from human artist...
2  mit researchers develop new ai safety framewor...
3  aipowered recommendation systems are actually ...
4  job interviews evaluated by ai sounds like dis...

              processed_text
0  ai algorithm perpetuating systemic racism crim...
1      ai generated art stealing human artist theft
2  mit researcher develop new ai safety framework...
3  aipowered recommendation system actually getti...
4  job interview evaluated ai sound like discrimi...
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

```

#d. Sentiment Analysis

```

from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
import pandas as pd

# Download VADER lexicon (it will check if already downloaded)
nltk.download('vader_lexicon')

sia = SentimentIntensityAnalyzer()

def get_vader_sentiment(text):
    if pd.isna(text) or not isinstance(text, str) or not text.strip(): # Handle
        return 'Neutral'
    score = sia.polarity_scores(text)['compound']
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

df['vader_sentiment'] = df['processed_text'].apply(get_vader_sentiment)

```

```

print("Original Sentiment vs. VADER Sentiment Samples:")
print(df[['sentiment', 'processed_text', 'vader_sentiment']].head())

print("\nValue Counts for Original Sentiment:")
print(df['sentiment'].value_counts())

print("\nValue Counts for VADER Sentiment:")
print(df['vader_sentiment'].value_counts())

```

Original Sentiment vs. VADER Sentiment Samples:

	sentiment	processed_text	vader_sentiment
0	Negative	ai algorithm perpetuating systemic racism crim...	Neg
1	Negative	ai generated art stealing human artist theft	Neg
2	Neutral	mit researcher develop new ai safety framework...	Pos
3	Positive	aipowered recommendation system actually getti...	Pos
4	Negative	job interview evaluated ai sound like discrimi...	Pos

Value Counts for Original Sentiment:

```

sentiment
Negative    25
Neutral     25
Positive    25
Name: count, dtype: int64

```

Value Counts for VADER Sentiment:

```

vader_sentiment
Positive    36
Neutral     24
Negative    15
Name: count, dtype: int64
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!

```

#e. Visualization

```

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.countplot(x='vader_sentiment', data=df, palette='viridis', order=['Positive', 'Neutral', 'Negative'])
plt.title('Distribution of VADER Sentiment Scores')
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

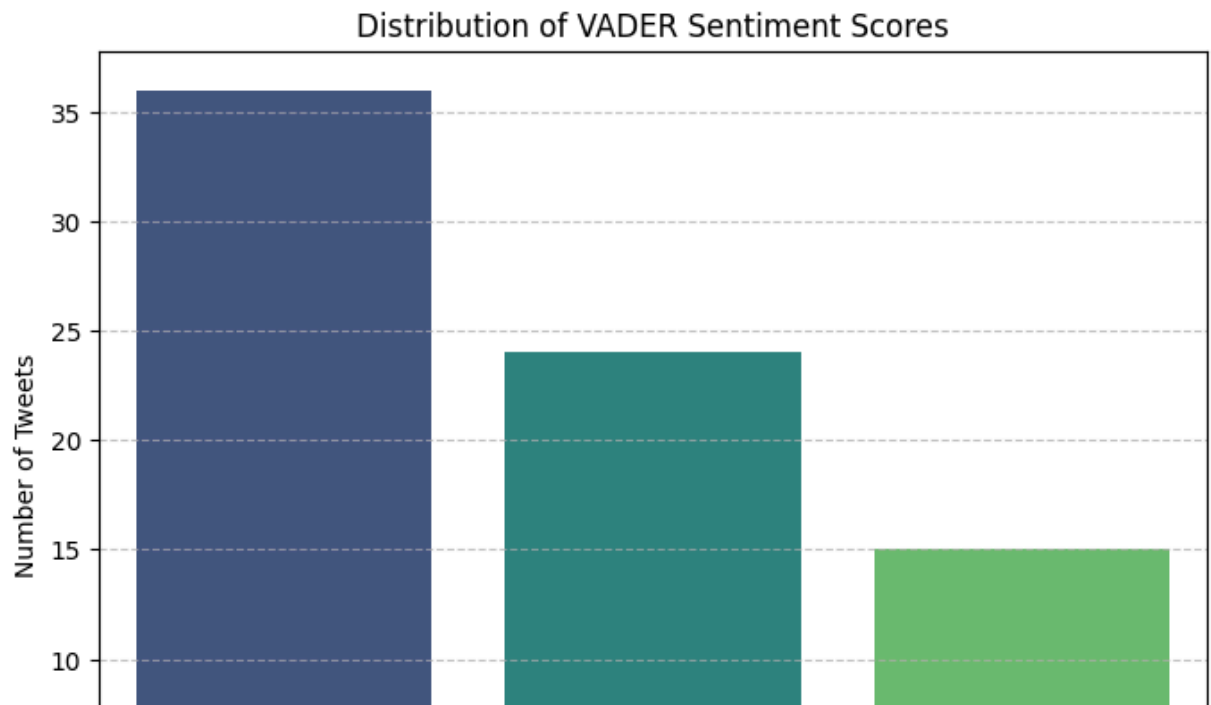
sentiment_percentage = df['vader_sentiment'].value_counts(normalize=True) * 100
print("\nPercentage Distribution of VADER Sentiment:")
print(sentiment_percentage)

```

```
/tmp/ipykernel_1416/1761342615.py:5: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed
```

```
sns.countplot(x='vader_sentiment', data=df, palette='viridis', order=['
```



#7. Sales Forecasting

#a. Load dataset

```
df=pd.read_csv('/content/sales_forecasting_data.csv')  
df
```

	date	day	month	month_name	year	quarter	week_number	day_of_week
0	2022-01-01	1	1	January	2022	1	52	Saturday
1	2022-01-01	1	1	January	2022	1	52	Saturday
2	2022-01-01	1	1	January	2022	1	52	Saturday
3	2022-01-01	1	1	January	2022	1	52	Saturday
4	2022-01-01	1	1	January	2022	1	52	Saturday
...	...	...	...	...	...	...	...	...
2919	2023-12-31	31	12	December	2023	4	52	Sunday
2920	2023-12-31	31	12	December	2023	4	52	Sunday
2921	2023-12-31	31	12	December	2023	4	52	Sunday
2922	2023-12-31	31	12	December	2023	4	52	Sunday
2923	2023-12-31	31	12	December	2023	4	52	Sunday

2924 rows × 19 columns

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Preprocessing data

```
df['date'] = pd.to_datetime(df['date'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2924 entries, 0 to 2923
Data columns (total 19 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   date                2924 non-null  datetime64[ns]
1   day                 2924 non-null  int64   
2   month               2924 non-null  int64   
3   month_name          2924 non-null  object  
4   year                2924 non-null  int64   
5   quarter             2924 non-null  int64   
```

```

6  week_number      2924 non-null  int64
7  day_of_week      2924 non-null  object
8  product          2924 non-null  object
9  category         2924 non-null  object
10 store            2924 non-null  object
11 quantity         2924 non-null  int64
12 unit_price       2924 non-null  float64
13 total_sales      2924 non-null  float64
14 discount_percent 2924 non-null  int64
15 is_promotion     2924 non-null  bool
16 cost             2924 non-null  float64
17 profit           2924 non-null  float64
18 profit_margin    2924 non-null  float64
dtypes: bool(1), datetime64[ns](1), float64(5), int64(7), object(5)
memory usage: 414.2+ KB

```

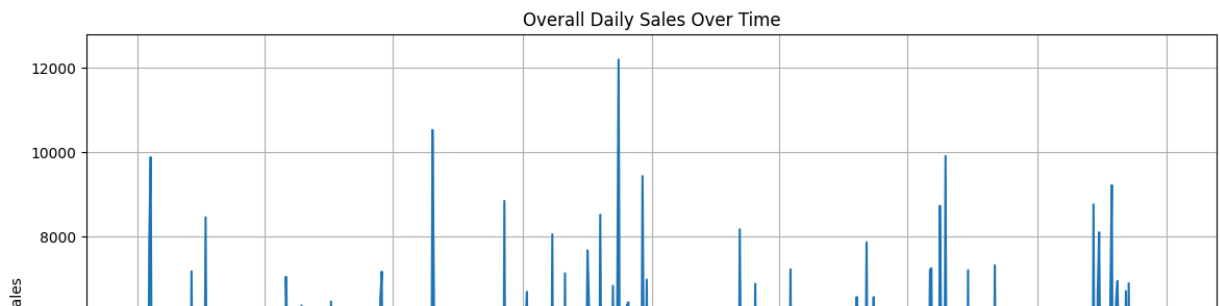
#b. Exploratory Data Analysis

```

daily_sales = df.groupby('date')['total_sales'].sum().reset_index()

plt.figure(figsize=(14, 7))
sns.lineplot(x='date', y='total_sales', data=daily_sales)
plt.title('Overall Daily Sales Over Time')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.grid(True)
plt.show()

```



To further investigate seasonality, let's look at sales trends by month and day of the week.

```

monthly_sales = df.groupby('month_name')['total_sales'].sum().reindex(['January',
                                'February', 'March', 'April', 'May', 'June',
                                'July', 'August', 'September', 'October', 'November', 'December'])

day_of_week_sales = df.groupby('day_of_week')['total_sales'].sum().reindex(['Monday',
                                    'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'])

plt.figure(figsize=(18, 6))

plt.subplot(1, 2, 1)
sns.barplot(x=monthly_sales.index, y=monthly_sales.values, palette='viridis')
plt.title('Total Sales by Month')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)

```

```
plt.subplot(1, 2, 2)
sns.barplot(x=day_of_week_sales.index, y=day_of_week_sales.values, palette='pl
plt.title('Total Sales by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

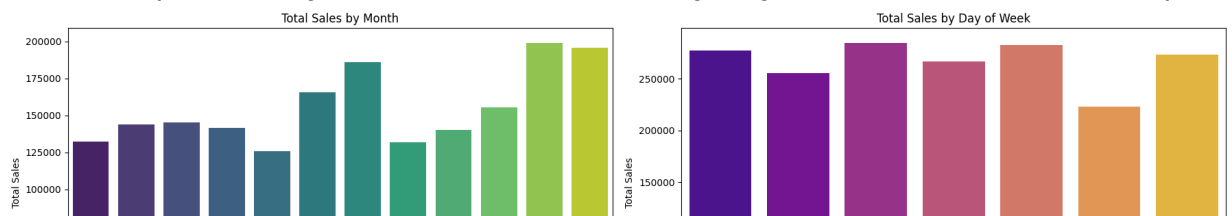
/tmp/ipykernel_1416/3706385297.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remov

```
sns.barplot(x=monthly_sales.index, y=monthly_sales.values, palette='vir
/tmp/ipykernel_1416/3706385297.py:15: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be remov

```
sns.barplot(x=day_of_week_sales.index, y=day_of_week_sales.values, pale
```



#c. Time Series Analysis

For time series analysis, we often need the time variable (date in this case) to be the index of our DataFrame. We will use the `daily_sales` DataFrame created earlier.

```
daily_sales_ts = daily_sales.set_index('date')
daily_sales_ts.index = pd.to_datetime(daily_sales_ts.index)
display(daily_sales_ts.head())
```

	total_sales 
date	
2022-01-01	851.88
2022-01-02	2347.21
2022-01-03	98.76
2022-01-04	4218.61
2022-01-05	826.28

✓ Check for Stationarity

Stationarity is a key concept in time series analysis. A stationary time series is one whose statistical properties (like mean, variance, autocorrelation) do not change over time. Many time series models, including ARIMA, assume stationarity. We'll use the Augmented Dickey-Fulle (ADF) test to check for it.

```
from statsmodels.tsa.stattools import adfuller

def check_stationarity(timeseries):
    # Dickey-Fuller test:
    print('Results of Dickey-Fuller Test:')
    dfctest = adfuller(timeseries, autolag='AIC')
    dfcoutput = pd.Series(dfctest[0:4], index=['Test Statistic', 'p-value', '#Lags
    for key,value in dfctest[4].items():
        dfcoutput['Critical Value (%s)'%key] = value
    print(dfcoutput)

    if dfcoutput['p-value'] <= 0.05 and dfcoutput['Test Statistic'] < dfcoutput['
    print("\nConclusion: The time series is likely stationary.")
    else:
        print("\nConclusion: The time series is likely NOT stationary. Differen

check_stationarity(daily_sales_ts['total_sales'])
```

```
Results of Dickey-Fuller Test:
Test Statistic          -19.777246
p-value                  0.000000
#Lags Used               1.000000
Number of Observations Used  728.000000
Critical Value (1%)      -3.439364
Critical Value (5%)      -2.865518
Critical Value (10%)     -2.568888
dtype: float64
```

Conclusion: The time series is likely stationary.

#d. Model building

✓ Train-Test Split for Time Series

```
train_size = int(len(daily_sales_ts) * 0.8)
train_data, test_data = daily_sales_ts[0:train_size], daily_sales_ts[train_size:]

print(f"Train data points: {len(train_data)}")
print(f"Test data points: {len(test_data)}")
display(train_data.head())
display(test_data.head())
```

Train data points: 584

Test data points: 146

total_sales 	
date	
2022-01-01	851.88
2022-01-02	2347.21
2022-01-03	98.76
2022-01-04	4218.61
2022-01-05	826.28
total_sales	
date	
2023-08-08	3711.23
2023-08-09	3020.34
2023-08-10	599.24
2023-08-11	2732.06
2023-08-12	1506.81

✓ ARIMA Model Training and Prediction

```
from statsmodels.tsa.arima.model import ARIMA

# Fit ARIMA model
# (p,d,q) order parameters: p=AR order, d=Differencing order, q=MA order
# Based on stationarity test, d=0 seems appropriate.
# p and q can be determined by ACF/PACF plots or auto_arima (which is not available)
# For demonstration, let's start with a common order like (5,0,0) or (1,0,0).
# We will use (5,0,0) as a starting point, implying an AR(5) model since the data is stationary

# Suppress ConvergenceWarning
import warnings
from statsmodels.tools.sm_exceptions import ConvergenceWarning
warnings.simplefilter('ignore', ConvergenceWarning)

arima_model = ARIMA(train_data['total_sales'], order=(5, 0, 0)) # Using AR(5)
arima_result = arima_model.fit()

print(arima_result.summary())

# Make predictions
start_index = len(train_data)
end_index = len(daily_sales_ts) - 1

predictions = arima_result.predict(start=start_index, end=end_index, dynamic=False)
```



```
# Create a DataFrame for predictions for easier plotting
# Correcting the creation of predictions_df
predictions_df = pd.DataFrame(predictions) # Create DataFrame directly from the
predictions_df.columns = ['predicted_sales'] # Rename the column
display(predictions_df.head())
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py
self._init_dates(dates, freq)
```

SARIMAX Results

```
=====
Dep. Variable:          total_sales      No. Observations:
Model:                ARIMA(5, 0, 0)    Log Likelihood      -527
Date:                 Sun, 24 May 2026  AIC                  1055
Time:                  14:01:18         BIC                  1058
Sample:               01-01-2022        HQIC                  1057
                   - 08-07-2023
```

Covariance Type: opg

```
=====
              coef      std err          z      P>|z|      [0.025      0
-----
const      2542.4509    112.451     22.609     0.000    2322.052    276
ar.L1         0.0669     0.040     1.682     0.093     -0.011
ar.L2        -0.0493     0.045    -1.094     0.274     -0.138
ar.L3         0.0442     0.044     1.008     0.314     -0.042
ar.L4         0.0119     0.039     0.304     0.761     -0.065
ar.L5        -0.0516     0.044    -1.184     0.236     -0.137
sigma2      4.072e+06    2.48e+05    16.441     0.000    3.59e+06    4.5
=====
```

```
Ljung-Box (L1) (Q):          0.00   Jarque-Bera (JB):
Prob(Q):                    0.99   Prob(JB):
Heteroskedasticity (H):      0.99   Skew:
Prob(H) (two-sided):         0.94   Kurtosis:
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (co

predicted_sales 

2023-08-08	2458.069476
2023-08-09	2667.383619
2023-08-10	2623.498883
2023-08-11	2515.808664
2023-08-12	2589.873076

✓ e. Evaluation

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

# Calculate evaluation metrics
mse = mean_squared_error(test_data['total_sales'], predictions)
rmse = np.sqrt(mse)
mae = mean_absolute_error(test_data['total_sales'], predictions)

print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
```

Mean Squared Error (MSE): 3860192.54
Root Mean Squared Error (RMSE): 1964.74
Mean Absolute Error (MAE): 1553.75

✓ Visualize Predictions

```
plt.figure(figsize=(15, 7))
plt.plot(train_data.index, train_data['total_sales'], label='Train Sales')
plt.plot(test_data.index, test_data['total_sales'], label='Actual Test Sales')
plt.plot(predictions_df.index, predictions_df['predicted_sales'], label='Predicted Sales')
plt.title('Sales Forecasting with ARIMA')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.legend()
plt.grid(True)
plt.show()
```

